



Universidad Nacional Autónoma de México

Facultad de Ciencias

Ciencias de la Computación

Práctica 5: Tabla de Símbolos y Ámbitos

Compiladores

Semestre 2027-1

Equipo: 02

Integrantes: Flores Galeana Daniela - 321162342
Rivera Machuca Gabriel Eduardo - 321057608

Profesora: L. en C.C. Yessica Janeth Pablo Martínez
Ayudante: Pedro Yamil Salazar González

Fecha de entrega: 23 de Octubre de 2026

Índice

1. Objetivo **2**

2. Evolución respecto de la Práctica 4 **2**
2.1. Módulos agregados y modificados 2

3. Diseño de la Tabla de Símbolos y Ámbitos **2**

4. Recorrido del AST y resolución de nombres **3**

5. Propiedad y memoria **3**

6. Diagnósticos y recuperación **3**

7. Resultados y pruebas **4**

8. Problemas encontrados **4**

9. División del trabajo **4**

10. Uso de herramientas de Inteligencia Artificial **5**

11. Conclusiones **5**

12. Referencias **6**

1. Objetivo

El objetivo principal de esta quinta práctica fue implementar la primera etapa del analizador semántico del lenguaje MiniC mediante el recorrido del Árbol de Sintaxis Abstracta (AST), la construcción de una tabla de símbolos y el manejo de ámbitos anidados. Su propósito fundamental es verificar que las construcciones sintácticamente válidas respeten las reglas de significado del lenguaje, detectando identificadores no declarados, redeclaraciones dentro del mismo ámbito, violaciones de visibilidad y controlando adecuadamente el ocultamiento de nombres (*shadowing*).

2. Evolución respecto de la Práctica 4

2.1. Módulos agregados y modificados

Se incorporó un nuevo conjunto de módulos dedicados al análisis semántico, la gestión de símbolos y el control de ámbitos léxicos, estructurados en los directorios `src/semantic/` e `include/semantic/` (incluyendo `symbol.c/h`, `scope.c/h`, `symbol_table.c/h` y `semantic.c/h`).

El controlador principal (`main.c`) fue actualizado para coordinar el flujo completo: tras superar exitosamente el análisis léxico y sintáctico y construir el AST, se inicializa el analizador semántico. Asimismo, la salida estándar por éxito se ajustó para emitir exactamente `Programa semanticamente correcto.` y terminar con código `0`, sustituyendo la impresión previa del AST de la Práctica 4.

3. Diseño de la Tabla de Símbolos y Ámbitos

El análisis semántico se apoya en una estructura jerárquica de ámbitos léxicos y una tabla de símbolos centralizada:

- **Símbolos (`Symbol`):** Cada variable declarada se representa mediante una estructura que almacena una copia propia de su nombre (`strdup`), su tipo declarado (`TYPE_INT`, `TYPE_BOOL`), su nivel de ámbito, la línea y columna exactas de su declaración, y una referencia prestada (no propietaria) al nodo de declaración del AST.
- **Ámbitos (`Scope`):** Representados como contenedores dinámicos de símbolos con redimensionamiento automático mediante `realloc`. Cada ámbito mantiene un puntero a su ámbito padre (`parent`) para establecer la cadena de visibilidad léxica y un registro de propiedad global (`next_owned`).
- **Tabla de Símbolos (`SymbolTable`):** Administra la pila de ámbitos activos (`current_scope`) y la lista completa de ámbitos creados para garantizar su liberación segura al finalizar el análisis.

4. Recorrido del AST y resolución de nombres

El analizador semántico recorre el AST de manera recursiva descendente mediante funciones visitantes especializadas por tipo de nodo (`visit_program`, `visit_block`, `visit_variable_declaration`, `visit_assignment`, `visit_print_statement`, `visit_if_statement`, `visit_while_statement`, `visit_expression`).

Las reglas principales aplicadas durante el recorrido son:

- **Orden secuencial:** Las sentencias se procesan en el orden en que aparecen. Una variable se registra en la tabla de símbolos inmediatamente al encontrar su declaración y antes de recorrer su inicializador, permitiendo construcciones como `int x = x;`.
- **Búsqueda Local y Visible:** La búsqueda local (`symbol_table_lookup_current`) se emplea al declarar variables para detectar redeclaraciones en el mismo ámbito[cite: 42, 52]. La búsqueda visible (`symbol_table_lookup`) asciende desde el ámbito actual a través de los padres hasta el ámbito global para resolver el uso de identificadores.
- **Ámbitos y Shadowing:** Cada bloque delimitado por llaves, así como las ramas de control, gestiona la creación y destrucción de ámbitos hijos, permitiendo que un ámbito interno oculte variables exteriores sin generar errores de redeclaración.

5. Propiedad y memoria

Se estableció una política estricta de propiedad y gestión de recursos:

- El módulo de símbolos copia los nombres mediante `strdup`, mientras que el nodo de declaración del AST es estrictamente un recurso prestado que nunca es liberado por la tabla de símbolos.
- La tabla de símbolos adquiere la propiedad exclusiva del símbolo únicamente cuando la inserción es exitosa (`SYMBOL_TABLE_OK`); en caso de duplicado o fallo interno, el código llamador es responsable de liberarlo.
- La función `semantic_analyzer_destroy` (junto con `symbol_table_destroy` y `scope_destroy`) garantiza la liberación recursiva y segura de todos los ámbitos y símbolos acumulados, previniendo fugas de memoria incluso tras inicializaciones parciales o fallos internos.

6. Diagnósticos y recuperación

Cuando se detecta una infracción semántica (como una variable no declarada o una redeclaración), se emite un diagnóstico estricto hacia `stderr` indicando la categoría, posición exacta (línea y columna), causa y el nombre del identificador involucrado. El

analizador acumula errores semánticos recuperables para reportar múltiples fallos en una sola ejecución. Ante fallos internos (como errores de asignación de memoria), se aborta el recorrido de manera controlada y se finaliza con un código distinto de cero.

7. Resultados y pruebas

El sistema fue evaluado rigurosamente utilizando la batería de pruebas públicas y casos propios desarrollados en la carpeta `tests/equipo/`, cubriendo:

- Programas vacíos y declaraciones globales simples y múltiples.
- Uso correcto de variables y detección de variables no declaradas en asignaciones, expresiones y sentencias `print`.
- Detección de redeclaraciones en el ámbito global y dentro de bloques anidados.
- Validación de visibilidad y ocultamiento de nombres (*shadowing*) en múltiples niveles.
- Acceso a variables fuera de su ámbito tras abandonar un bloque.
- Acumulación de múltiples errores semánticos simultáneos.

El compilador respondió con los códigos de salida apropiados (0 para programas semánticamente válidos y 1 para entradas con errores), dejando `stdout` vacío en caso de fallos. Asimismo, se verificó mediante herramientas de depuración la ausencia total de fugas de memoria.

8. Problemas encontrados

Durante el desarrollo de la tabla de símbolos se presentaron desafíos sutiles relacionados con la administración de los punteros a los ámbitos padre durante las operaciones de entrada y salida de bloques anidados, así como la correcta propagación del estado de error interno. Estos aspectos se resolvieron implementando un registro explícito de ámbitos poseídos (`owned_scopes`) y asegurando que las salidas de ámbito se ejecuten de forma garantizada incluso cuando ocurren errores semánticos en las sentencias internas de un bloque.

9. División del trabajo

- **Gabriel Eduardo Rivera Machuca:** Diseño e implementación de la estructura de símbolos (`symbol.c/h`), la gestión de ámbitos anidados (`scope.c/h`), la tabla de símbolos (`symbol_table.c/h`) y las pruebas unitarias de aislamiento y visibilidad.

- **Daniela Flores Galeana:** Implementación del analizador semántico (`semantic.c/h`), desarrollo de los visitantes recursivos del AST, integración con el controlador principal (`main.c`), diseño del sistema de diagnósticos semánticos y validación con la suite de pruebas y herramientas de memoria.

Ambos integrantes colaboraron activamente en la revisión de la arquitectura, pruebas de integración y depuración general del compilador.

10. Uso de herramientas de Inteligencia Artificial

Para esta práctica se utilizó Gemini (Google) como herramienta de apoyo técnico y consulta.

- **Apoyo en el reporte:** Al compilar mi archivo .tex me sale el siguiente error y no se muestran las imagenes como quiero, como lo soluciono en terminal para debian13.
- **Revisión y comprobación:** Estoy programando un compilador y tengo el siguiente case dentro de un switch

  switch(node-¿type)

  case AST$_I DENTIFIER$:

  const char *name = node-¿data.identifier.name;

  Symbol *sym = symbol$_t able_l ookup(analyzer- > symbol_t able, name)$;

if(!sym)

char buf[256];

snprintf(buf, sizeof(buf), "la variable '

report$_s emantic_e rror(analyzer, node- > line, node- > column, buf)$;

break;

Y me da el siguiente error: src/semantic/semantic.c: In function 'visit$_e xpression'$:

src/semantic/semantic.c:177:13: warning: a label can only be part of a statement and a declaration is not a statement [-Wpedantic]

177 — const char *name = node-¿data.identifier.name;

Cómo lo soluciono?

11. Conclusiones

La implementación de la tabla de símbolos y el análisis semántico marcó una evolución crucial en el desarrollo del compilador, transformándolo de un mero verificador sintáctico a un sistema capaz de comprender el significado contextual del código fuente. El dominio del manejo de ámbitos anidados léxicos, la visibilidad de identificadores y la

estricta gestión dinámica de memoria en C proporcionan la base sólida requerida para afrontar con éxito la siguiente etapa de verificación de tipos y generación de código intermedio.

12. Referencias

Referencias

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2da ed.). Pearson.

[2] Fischer, C. N., Cytron, R. K., & LeBlanc, R. J. (2009). *Crafting a compiler*. Pearson.

[3] Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning.

[4] Pablo Martínez, Y. J. (2026). *Guía de integración y especificaciones de la Práctica 5*. Facultad de Ciencias, UNAM.